



@m1rko

2 апр 2019 в 21:12

Word2vec в картинках



14 мин

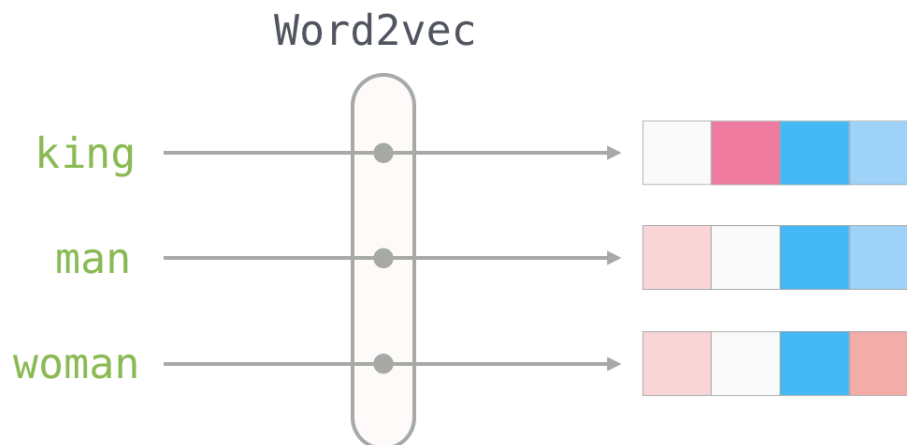


177K

Машинное обучение*

[Перевод](#)

Автор оригинала: Jay Alammar



«Во всякой вещи скрыт узор, который есть часть Вселенной. В нём есть симметрия, элегантность и красота — качества, которые прежде всего схватывает всякий истинный художник, запечатлевающий мир. Этот узор можно уловить в смене сезонов, в том, как струится по склону песок, в перепутанных ветвях креозотового кустарника, в узоре его листа.

Мы пытаемся скопировать этот узор в нашей жизни и нашем обществе и потому любим ритм, песню, танец, различные радующие и утешающие нас формы. Однако можно разглядеть и опасность, таящуюся в поиске абсолютного совершенства, ибо очевидно, что совершенный узор — неизменен. И, приближаясь к совершенству, всё сущее идёт к смерти» — *Дюна* (1965)

Я считаю, что концепция вложений (embeddings) — одна из самых замечательных идей в машинном обучении. Если вы когда-нибудь использовали Siri, Google Assistant, Alexa, Google Translate или даже клавиатуру смартфона с предсказанием следующего слова, то уже работали с моделью обработки естественного языка на основе вложений. За последние десятилетия произошло значительное развитие этой концепции для нейронных моделей (последние разработки включают контекстуализированные вложения слов в передовых моделях, таких как BERT и GPT2).

Word2vec — метод эффективного создания вложений, разработанный в 2013 году. Кроме работы со словами, некоторые его концепции оказались эффективны в разработке рекомендательных механизмов и придании смысла данным даже в коммерческих, неязыковых задачах. Эту технологию применили в своих движках рекомендаций такие компании, как Airbnb, Alibaba, Spotify и Anghami.

В этой статье рассмотрим концепцию и механику генерации вложений с помощью word2vec. Начнём с примера, чтобы ознакомиться с тем, как представлять объекты в векторном виде. Вы знаете, насколько много о вашей личности может сказать список из пяти чисел (вектор)?

Встраивание личностей: что вы из себя представляете?

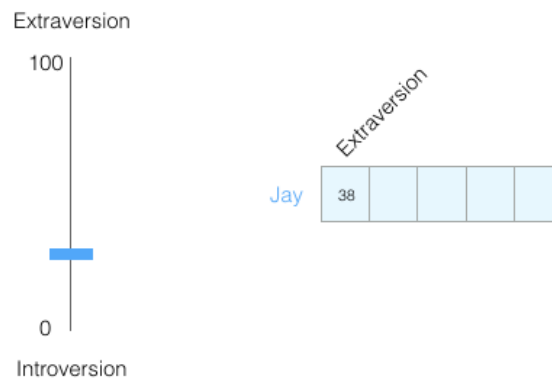
«Я даю тебе хамелеона Пустыни; его способность сливаться с песком скажет тебе всё, что надлежит знать о корнях экологии и основаниях сохранения личности». — *Дети Дюны*

По шкале от 0 до 100 у вас интровертный или экстравертный тип личности (где 0 — максимально интровертный тип, а 100 — максимально экстравертный)? Вы когда-нибудь проходили личностный тест: например, MBTI, а ещё лучше «большую пятёрку»? Вам дают список вопросов, а затем оценивают по нескольким осям, в том числе интровертность/экстравертность.

Openness to experience	79 out of 100
Agreeableness	75 out of 100
Conscientiousness	42 out of 100
Negative emotionality	50 out of 100
Extraversion	58 out of 100

Пример результатов теста «большая пятёрка». Он действительно многое говорит о личности и способен прогнозировать учебный, личный и профессиональный успех. Например, [здесь можно его пройти](#)

Предположим, я набрал 38 из 100 по оценке интроверсии/экстраверсии. Это можно изобразить следующим образом:

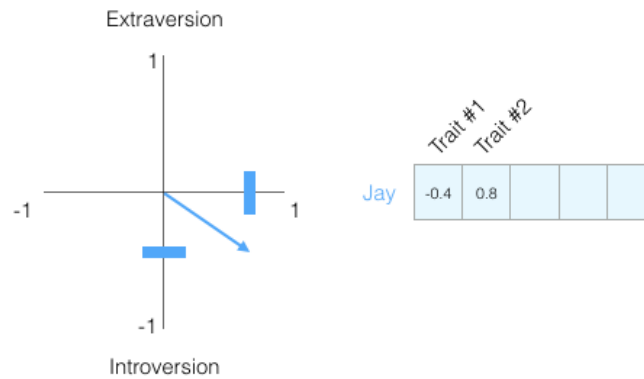


Или на шкале от -1 до +1:



Насколько хорошо мы узнаем человека только по этой оценке? Не особо. Люди — сложные

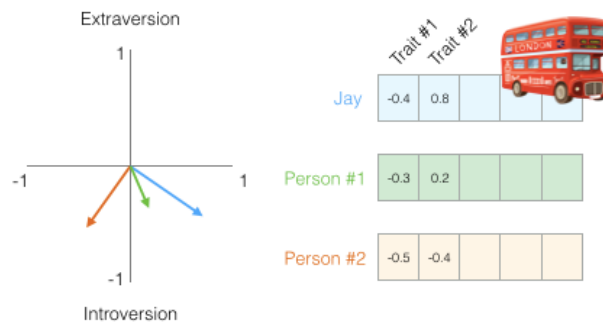
существа. Поэтому добавим ещё одно измерение: ещё одну характеристику из теста.



Можно представить эти два измерения как точку на графике или, ещё лучше, как вектор от начала координат до этой точки. Существуют великолепные инструменты для работы с векторами, которые очень скоро пригодятся

Я не показываю, какие черты личности мы наносим на график, чтобы вы не привязывались к конкретным чертам, а сразу понимали векторное представление личности человека в целом.

Теперь можно сказать, что этот вектор частично отражает мою личность. Это полезное описание, если сравнить разных людей. Допустим, меня сбил красный автобус, и нужно заменить меня похожей личностью. Кто из двух людей на следующем графике больше на меня похож?



При работе с векторами сходство обычно вычисляется по коэффициенту Отиаи (геометрический коэффициент):

$$\text{cosine_similarity}(\text{Jay}, \text{Person \#1}) = 0.87 \quad \checkmark$$

$$\text{cosine_similarity}(\text{Jay}, \text{Person \#2}) = -0.20$$

Человек №1 больше похож на меня по характеру. Векторы в одном направлении (длина также важна) дают больший коэффициент Отиаи

Опять же, двух измерений недостаточно для оценки людей. Десятилетия развития психологической науки привели к созданию теста на пять основных характеристик личности (с множеством дополнительных). Итак, давайте используем все пять измерений:

	Trait #1	Trait #2	Trait #3	Trait #4	Trait #5
Jay	-0.4	0.8	0.5	-0.2	0.3
Person #1	-0.3	0.2	0.3	-0.4	0.9
Person #2	-0.5	-0.4	-0.2	0.7	-0.1

Проблема с пятью измерениями в том, что уже не получится начертить аккуратные стрелки в 2D. Это общая проблема в машинном обучении, где часто приходится работать в многомерном пространстве. Хорошо, что геометрический коэффициент работает с любым количеством измерений:

$$\text{cosine_similarity}(\text{Jay}, \text{Person \#1}) = 0.66 \quad \checkmark$$

$$\text{cosine_similarity}(\text{Jay}, \text{Person \#2}) = -0.37$$

Геометрический коэффициент работает для любого количества измерений. По пяти измерениям результат гораздо точнее

В конце этой главы хочу повторить две главные идеи:

1. Людей (и другие объекты) можно представить в виде числовых векторов (что отлично подходит для машин!).
2. Мы можем легко вычислить, насколько похожи векторы.

1- We can represent things (and people) as vectors of numbers (Which is great for machines!)

Jay	-0.4	0.8	0.5	-0.2	0.3
-----	------	-----	-----	------	-----

2- We can easily calculate how similar vectors are to each other

The people most similar to Jay are:

cosine_similarity ▼

Person #1	0.86
Person #2	0.5
Person #3	-0.20

Встраивание слов

«Дар слов — это дар обмана и иллюзий». — *Дети Дюны*

С этим пониманием перейдём к векторным представлениям слов, полученным в результате обучения (их также называют вложениями) и посмотрим на их интересные свойства.

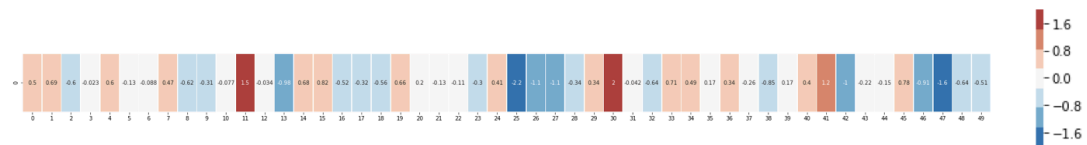
Вот вложение для слова «король» (вектор GloVe, обученный на Википедии):

[0.50451 , 0.68607 , -0.59517 , -0.022801, 0.60046 , -0.13498 , -0.08813 , 0.47377 ,
 -0.61798 , -0.31012 , -0.076666, 1.493 , -0.034189, -0.98173 , 0.68229 , 0.81722 ,
 -0.51874 , -0.31503 , -0.55809 , 0.66421 , 0.1961 , -0.13495 , -0.11476 , -0.30344 ,
 0.41177 , -2.223 , -1.0756 , -1.0783 , -0.34354 , 0.33505 , 1.9927 , -0.04234 ,
 -0.64319 , 0.71125 , 0.49159 , 0.16754 , 0.34344 , -0.25663 , -0.8523 , 0.1661 ,
 0.40102 , 1.1685 , -1.0137 , -0.21585 , -0.15155 , 0.78321 , -0.91241 , -1.6106 ,
 -0.64426 , -0.51042]

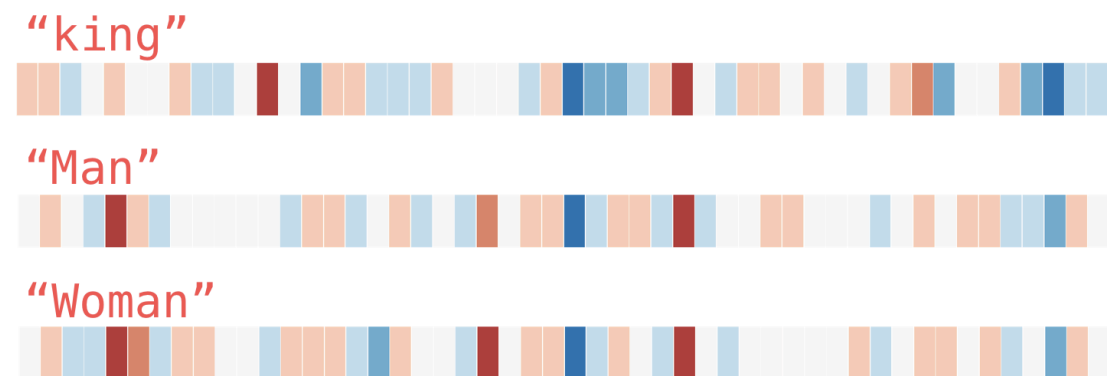
Мы видим список из 50 чисел, но по ним трудно что-то сказать. Давайте их визуализируем, чтобы сравнить с другими векторами. Поместим числа в один ряд.:



Раскрасим ячейки по их значениям (красный для близких к 2, белый для близких к 0, синий для близких к -2):

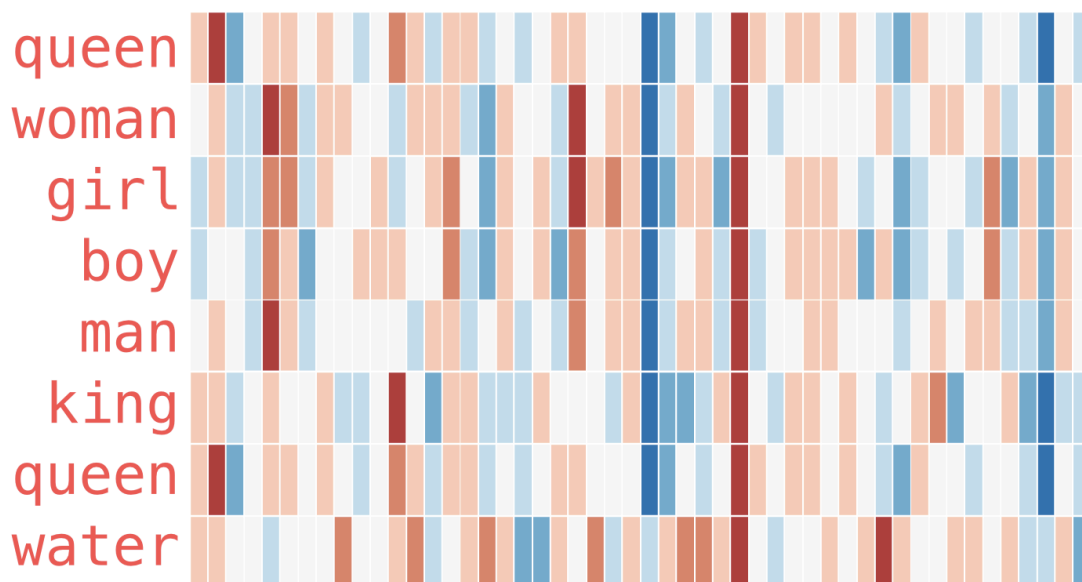


Теперь забудем о числах, и только по цветам противопоставим «короля» с другими словами:



Видите, что «мужчина» и «женщина» гораздо ближе друг к другу, чем к «королю»? Это о чём-то говорит. Векторные представления захватывают довольно много информации/значения/ассоциаций этих слов.

Вот ещё один список примеров (сравните столбцы с похожими цветами):



Можно заметить несколько вещей:

1. Через все слова проходит одна красная колонка. То есть эти слова похожи в этом конкретном измерении (и мы не знаем, что в нём закодировано).
2. Вы можете увидеть, что «женщина» и «девушка» во многом похожи. То же самое с «мужчиной» и «мальчиком».
3. «Мальчик» и «девочка» тоже похожи в некоторых измерениях, но отличаются от «женщины» и «мужчины». Может быть, это закодировано смутное представление о молодости? Вероятно.
4. Всё, кроме последнего слова — это представления людей. Я добавил объект (воду), чтобы показать различия между категориями. Например, вы можете увидеть, как синий столбец идёт вниз и останавливается перед вектором «воды».
5. Есть чёткие измерения, где «король» и «королева» похожи друг на друга и отличаются от всех остальных. Может, там закодирована расплывчатая концепция королевской власти?

Аналогии

«Слова выносят любую нагрузку, какую мы только пожелаем. Всё, что для этого требуется, — это соглашение о традиции, согласно которой мы строим понятия». — *Бог-император Дюны*

Знаменитые примеры, которые показывают невероятные свойства вложений, — понятие аналогий. Мы можем складывать и вычитать векторы слов, получая интересные результаты. Самый известный пример — формула «король – мужчина + женщина»:

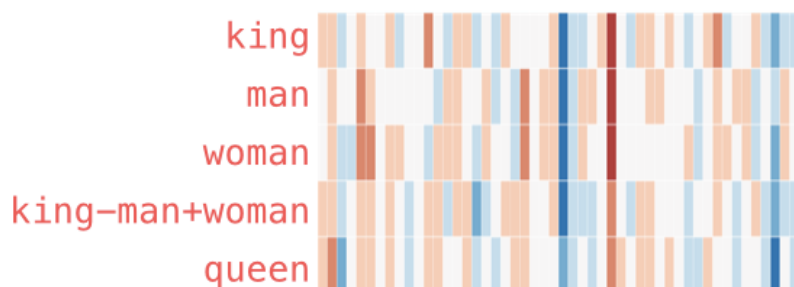
```
model.most_similar(positive=["king", "woman"], negative=["man"])

[('queen', 0.8523603677749634),
 ('throne', 0.7664333581924438),
 ('prince', 0.7592144012451172),
 ('daughter', 0.7473883032798767),
 ('elizabeth', 0.7460219860076904),
 ('princess', 0.7424570322036743),
 ('kingdom', 0.7337411642074585),
 ('monarch', 0.721449077129364),
 ('eldest', 0.7184862494468689),
 ('widow', 0.7099430561065674)]
```

Используя библиотеку *Gensim* в *python*, мы можем складывать и вычитать векторы слов, а библиотека найдёт слова, самые близкие к результирующему вектору. На изображении показан список наиболее похожих слов, каждое с коэффициентом геометрического сходства

Визуализируем эту аналогию, как раньше:

king - man + woman $\sim$ queen



Полученный вектор от вычисления «король-мужчина+женщина» не совсем равен «королеве», но это наиболее близкий результат из 400 000 вложенных слов в наборе данных

Рассмотрев вложения слов, давайте изучим, как происходит обучение. Но прежде чем перейти к word2vec, нужно взглянуть на концептуального предка вложений слов: нейронную модель языка.

Модель языка

«Пророк не подвержен иллюзиям прошлого, настоящего или будущего. **Фиксированность языковых форм определяет такие линейные различия.** Пророки же держат в руках ключ к замку языка. Для них физический образ остаётся всего лишь физическим образом и не более того.

Их вселенная не имеет свойств механической вселенной. Линейная последовательность событий полагается наблюдателем. Причина и следствие? Речь идёт совершенно о другом. Пророк высказывает судьбоносные слова. Вы видите отблеск события, которое должно случиться «по логике вещей». Но пророк мгновенно освобождает энергию бесконечной чудесной силы. Вселенная претерпевает духовный сдвиг». — *Бог-император Дюны*

Один из примеров NLP (обработка естественного языка) — функция предсказания следующего слова на клавиатуре смартфона. Миллиарды людей используют её сотни раз в день.



Предсказание следующего слова — подходящая задача для *модели языка*. Она может взять список слов (скажем, два слова) и попытаться предсказать следующее.

На скриншоте сверху модель взяла эти два зелёных слова (thou shalt) и вернула список вариантов (наибольшая вероятность для слова not):

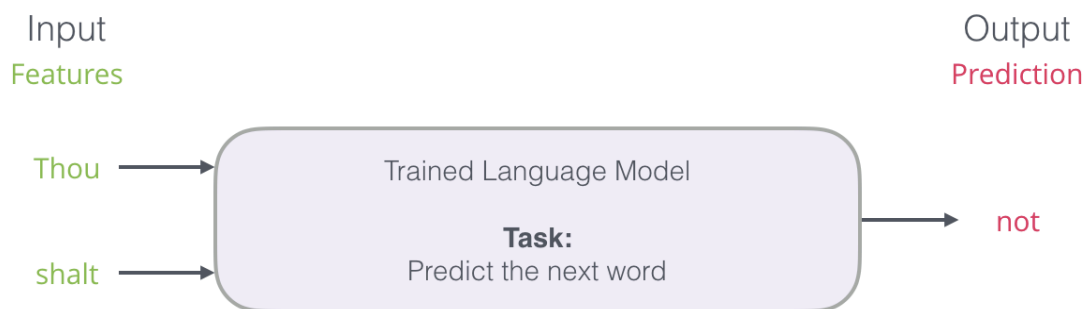
input/feature #1

input/feature #2

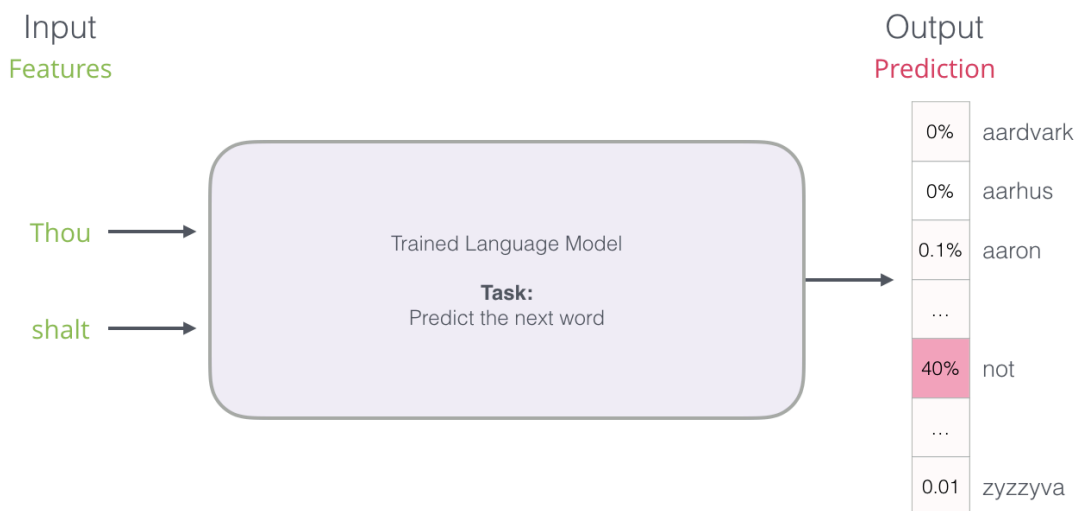
output/label

Thou shalt

Мы можем представить модель в виде чёрного ящика:

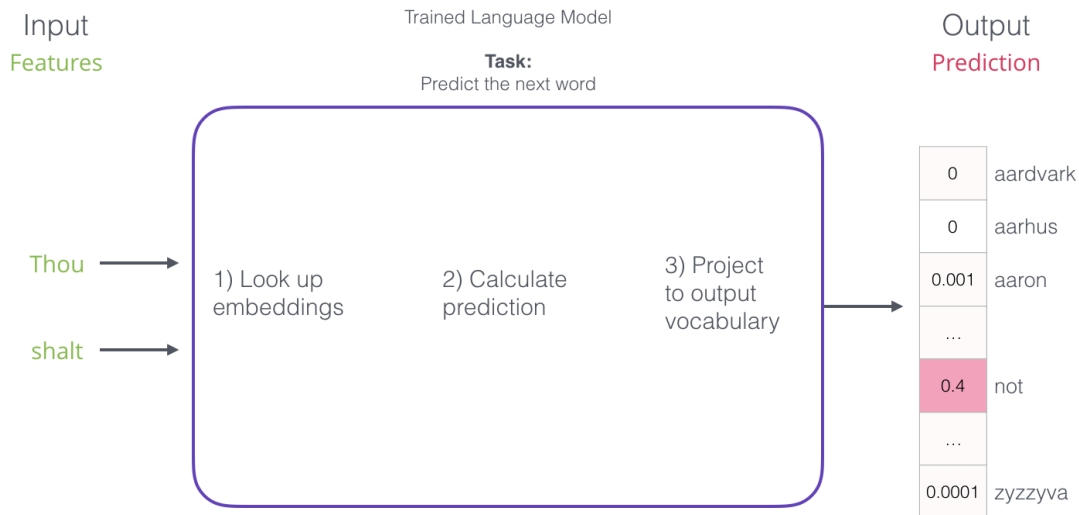


Но на практике модель выдаёт не одно слово. Она выводит оценку вероятности фактически для всех известных слов («словарь» модели варьируется от нескольких тысяч до более миллиона слов). Затем приложение клавиатуры находит слова с самыми высокими баллами и показывает их пользователю.

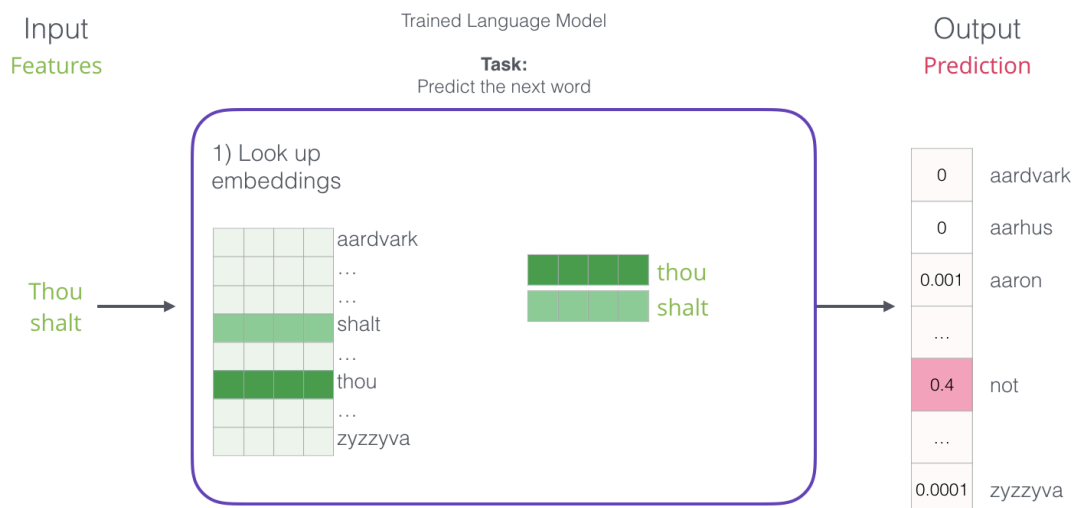


Нейронная модель языка выдаёт вероятность всех известных слов. Мы указываем вероятность в процентах, но в результирующем векторе 40% будет представлено как 0.4

После обучения первые нейронные модели (Bengio 2003) рассчитывали прогноз в три этапа:



Первый шаг для нас самый актуальный, поскольку мы обсуждаем вложения. В результате обучения создаётся матрица с вложениями всех слов нашего словаря. Для получения результата мы просто ищем вложения входных слова и запускаем прогнозирование:



Теперь давайте посмотрим на процесс обучения и узнаем, как создаётся эта матрица вложений.

Обучение модели языка

«Процесс нельзя понять посредством его прекращения. Понимание должно двигаться вместе с процессом, слиться с его потоком и течь вместе с ним» — *Дюна*

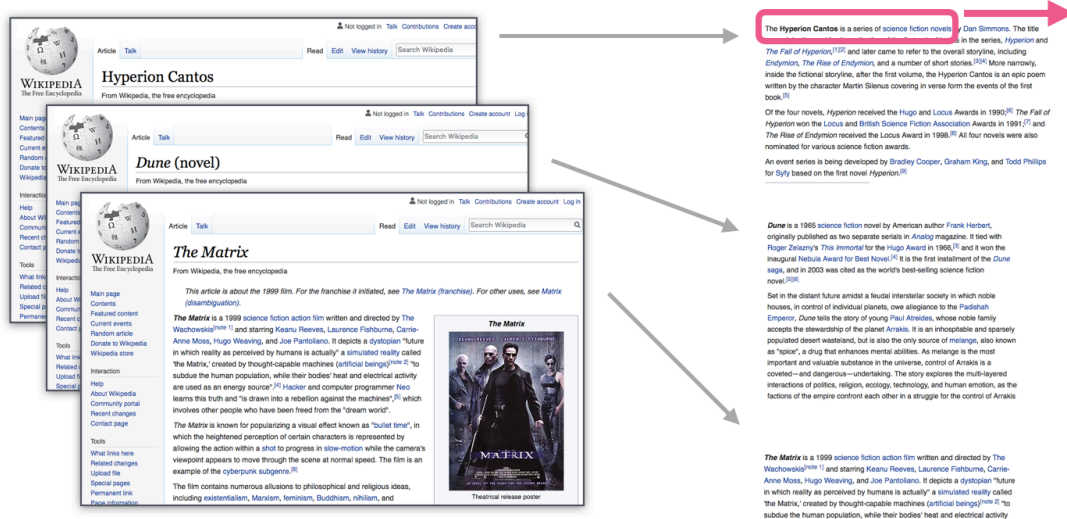
У языковых моделей огромное преимущество перед большинством других моделей машинного обучения: их можно обучать на текстах, которые у нас в изобилии. Подумайте обо всех книгах, статьях, материалах Википедии и других формах текстовых данных, которые у нас есть. Сравните с другими моделями машинного обучения, которым нужен ручной труд и специально собранные данные.

«Вы должны узнать слово по его компании» — Дж. Р. Фёрс

Вложения для слов вычисляются по окружающим словам, которые чаще появляются рядом.

Механика такая:

1. Получаем много текстовых данных (скажем, все статьи Википедии)
2. Устанавливаем окно (например, из трёх слов), которое скользит по всему тексту.
3. Скользящее окно генерирует образцы для обучения нашей модели



Когда это окно скользит по тексту, мы (фактически) генерируем набор данных, который затем используем для обучения модели. Для понимания посмотрим, как скользящее окно обрабатывает эту фразу:

«Да не построишь машины, наделённой подобием разума людского» — *Дюна*

Когда мы начинаем, окно располагается на первых трёх словах предложения:

Thou shalt not make a machine in the likeness of a human mind							
Sliding window across running text							
thou	shalt	not	make	a	machine	in	the ...
input 1				input 2		output	

Первые два слова принимаем за признаки, а третье слово — за метку:

Thou shalt not make a machine in the likeness of a human mind							
Sliding window across running text							
thou	shalt	not	make	a	machine	in	the ...
input 1				input 2		output	
thou				shalt		not	

Мы сгенерировали первый образец в наборе данных, который позже можно использовать для обучения языковой модели

Затем перемещаем окно в следующую позицию и создаём второй образец:

Thou shalt not make a machine in the likeness of a human mind

Sliding window across running text

thou	shalt	not	make	a	machine	in	the	...
thou	shalt	not	make	a	machine	in	the	

Dataset

input 1	input 2	output
thou	shalt	not
shalt	not	make

И довольно скоро у нас накапливается большой набор данных:

Thou shalt not make a machine in the likeness of a human mind

Sliding window across running text

thou	shalt	not	make	a	machine	in	the	...
thou	shalt	not	make	a	machine	in	the	
thou	shalt	not	make	a	machine	in	the	
thou	shalt	not	make	a	machine	in	the	
thou	shalt	not	make	a	machine	in	the	

Dataset

input 1	input 2	output
thou	shalt	not
shalt	not	make
not	make	a
make	a	machine
a	machine	in

На практике модели обычно обучаются непосредственно в процессе движения скользящего окна. Но логически фаза «генерации набора данных» отделена от фазы обучения. Помимо нейросетевых подходов, для обучения языковых моделей раньше часто использовался метод N-грамм (см. третью главу книги «Обработка речи и языка»). Чтобы увидеть разницу при переходе от N-грамм к нейронным моделям в реальных продуктах, вот сообщение 2015 года в блоге Swiftkey, разработчика моей любимой клавиатуры Android, который представляет свою нейронную модель языка и сравнивает её с предыдущей моделью N-грамм. Мне нравится этот пример, потому что он показывает, как алгоритмические свойства вложений можно описать маркетинговым языком.

Смотрим в обе стороны

«Парадокс — это признак того, что надо постараться рассмотреть, что за ним кроется. Если парадокс доставляет тебе беспокойство, то это значит, что ты стремишься к абсолюту. Релятивисты рассматривают парадокс просто как интересную, возможно, забавную, иногда страшную, мысль, но мысль весьма поучительную». *Бог-император Дюны*

С учётом всего сказанного ранее, заполните пробел:

Jay was hit by a _____

В качестве контекста есть пять предыдущих слов (и более раннее упоминание «автобуса»).

Уверен, что большинство из вас догадались, что здесь должен быть «автобус». Но если я дам вам ещё одно слово после пробела, это изменит ваш ответ?

Jay was hit by a _____ bus

Это полностью меняет ситуацию: теперь пропущенным словом, скорее всего, является «красный». Очевидно, что информационная ценность есть у слов и до, и после пробела. Оказывается, учёт в обе стороны (слева и справа) позволяет рассчитать более качественные вложения. Посмотрим, как в такой ситуации настроить обучение модели.

Skip-gram

«Когда абсолютно безошибочный выбор неизвестен, интеллект получает шанс поработать с ограниченными данными на арене, где ошибки не только возможны, но и необходимы». — Капитул Дюны

Кроме двух слов перед целевым, можно учитывать ещё два слова после него.

Jay was hit by a _____ bus in...

by	a	red	bus	in
----	---	-----	-----	----

Тогда набор данных для обучения модели будет выглядеть так:

input 1	input 2	input 3	input 4	output
by	a	bus	in	red

Это называется архитектурой CBOW (Continuous Bag of Words) и описывается в одном из документов word2vec [pdf]. Есть ещё другая архитектура, которая тоже демонстрирует отличные результаты, но устроена немного иначе: она пытается угадать соседние слова по текущему слову. Скользящее окно выглядит примерно так:

Jay was hit by a red bus in...



В зелёном слоте — входное слово, а каждое розовое поле представляет возможный выход

У розовых прямоугольников разные оттенки, потому что это скользящее окно фактически создает четыре отдельных образца в нашем наборе данных обучения:

Jay was hit **by a red bus in...**

by	a	red	bus	in
----	---	-----	-----	----

input	output
red	by
red	a
red	bus
red	in

Этот метод называется архитектурой **skip-gram**. Можно визуализировать скользящее окно следующим образом:

Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

input word	target word

В набор данных для обучения добавляется четыре следующих образца:

Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

input word	target word
not	thou
not	shalt
not	make
not	a

Затем перемещаем окно в следующую позицию:

Thou **shalt not make a machine** in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

input word	target word
not	thou
not	shalt
not	make
not	a

Которая генерирует ещё четыре примера:

Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine

Вскоре у нас гораздо больше образцов:

Thou shalt not make a machine in the likeness of a human mind

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

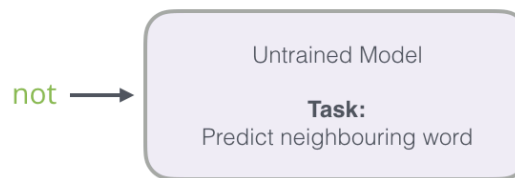
input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine
a	not
a	make
a	machine
a	in
machine	make
machine	a
machine	in
machine	the
in	a
in	machine
in	the
in	likeness

Пересмотр процесса обучения

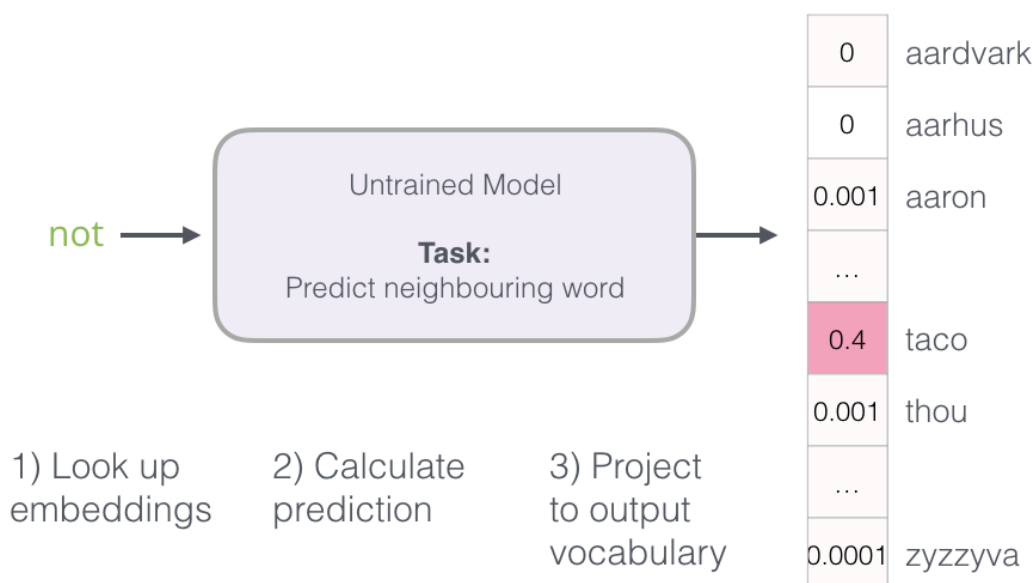
«Муад'Диб быстро учился потому, что прежде всего его научили тому, как надо учиться. Но самым первым уроком стало усвоение веры в то, что он может учиться, и это — основа всего. Просто поразительно, как много людей не верят в то, что могут учиться и научиться, и насколько больше людей считают, что учиться очень трудно». — *Дюна*

Теперь, когда у нас есть набор skip-gram, используем его для обучения базовой нейронной модели языка, которая предсказывает соседнее слово.

input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine
a	not
a	make
a	machine
a	in
machine	make
machine	a
machine	in
machine	the
in	a
in	machine
in	the
in	likeness



Начнём с первого образца в нашем наборе данных. Берём признак и отправляем его в необученную модель с просьбой предсказать соседнее слово.



Модель проходит три шага и выводит вектор предсказания (с вероятностью для каждого слова в словаре). Поскольку модель не обучена, на данном этапе её прогноз наверняка неправильный. Но это ничего. Мы знаем, какое слово она спрогнозирует — это результирующая ячейка в строке, которую мы в настоящее время используем для обучения модели:

Actual
Target

0
0
0
...
0
1
...
0

-

Model
Prediction

0	aardvark
0	aarhus
0.001	aaron
...	
0.4	taco
0.001	thou
...	
0.0001	zyzzyva

«Целевой вектор» — тот, в котором у целевого слова вероятность 1, а у всех остальных слов вероятность 0

Насколько ошиблась модель? Вычитаем вектор прогноза из целевого и получаем вектор ошибки:

Actual
Target

0
0
0
...
0
1
...
0

-

Model
Prediction

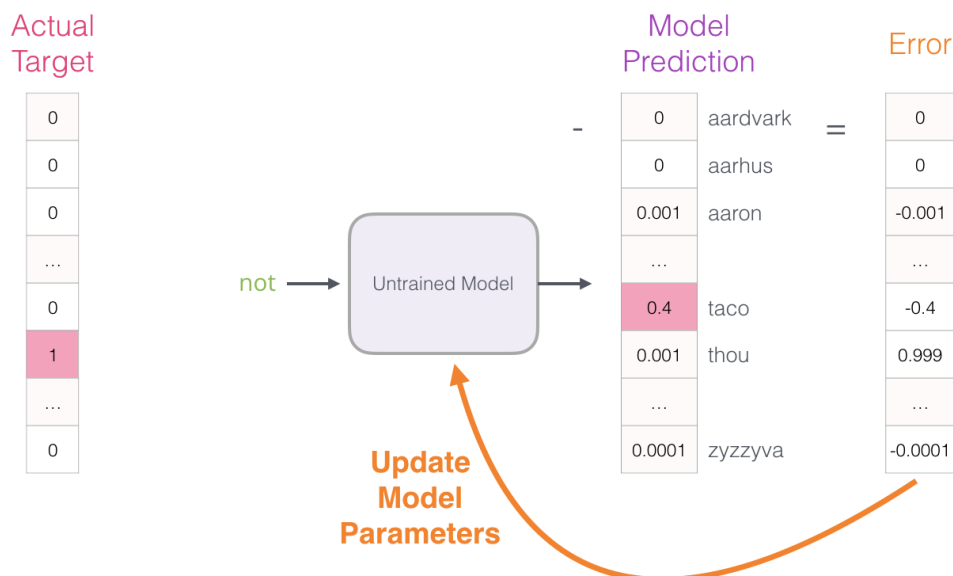
0	aardvark
0	aarhus
0.001	aaron
...	
0.4	taco
0.001	thou
...	
0.0001	zyzzyva

=

Error

0
0
-0.001
...
-0.4
0.999
...
-0.0001

Этот вектор ошибки теперь можно использовать для обновления модели, поэтому в следующий раз она с большей вероятностью выдаст точный результат на тех же входных данных.



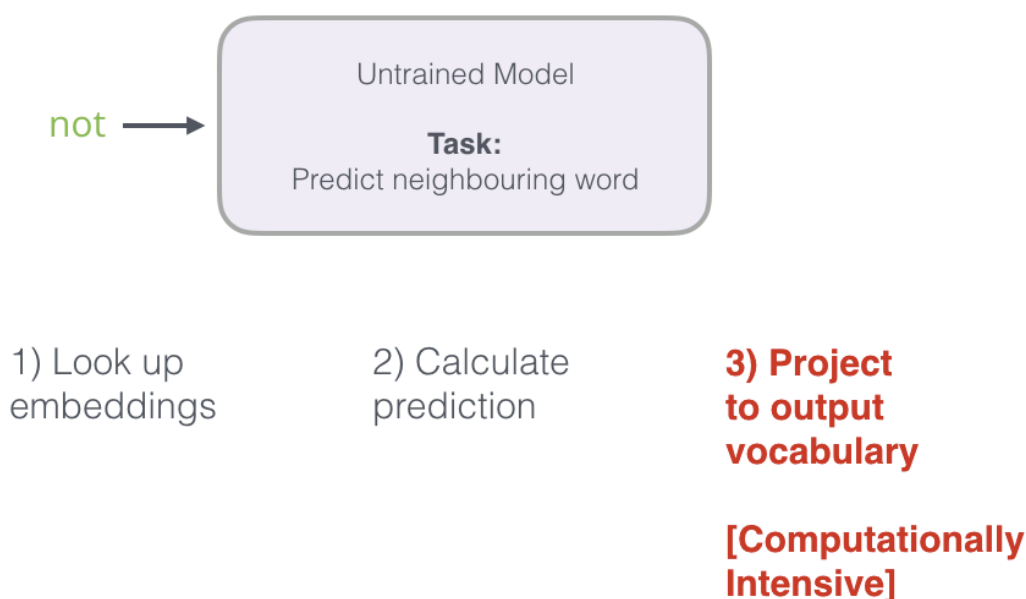
Здесь завершается первый этап обучения. Продолжаем делать то же самое со следующим образцом в наборе данных, а затем со следующим, пока не рассмотрим все образцы. Это конец первой эпохи обучения. Повторяем всё снова и снова в течение нескольких эпох, и в итоге получаем обученную модель: из неё можно извлечь матрицу вложений и использовать в любых приложениях.

Хотя мы многое узнали, но для полного понимания, как реально обучается word2vec, не хватает пары ключевых идей.

Отрицательный отбор

«Пытаться понять Муад'Дибя без того, чтобы понять его смертельных врагов — Харконненов, — это то же самое, что пытаться понять Истину, не поняв, что такое Ложь. Это — попытка познать Свет, не познав Тьмы. Это — невозможно». — *Дюна*

Вспомним три этапа, как нейронная модель вычисляет прогноз:



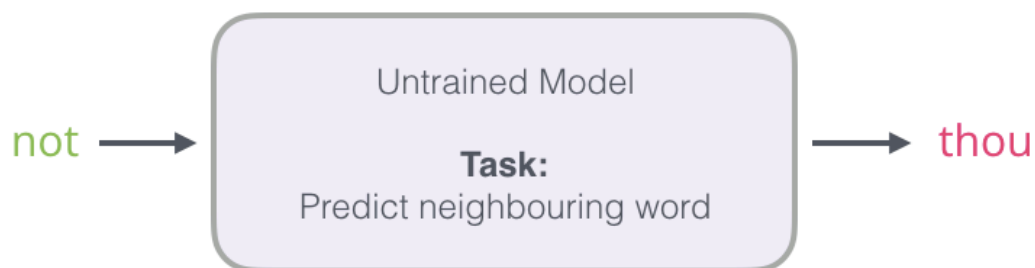
Третий шаг очень дорог с вычислительной точки зрения, особенно если делать его для каждой выборки в наборе данных (десятки миллионов раз). Нужно как-то повысить производительность.

Один из способов — разделить цель на два этапа:

1. Создать высококачественные вложения слов (без прогноза следующего слова).
2. Использовать эти высококачественные вложения для обучения языковой модели (для прогнозирования).

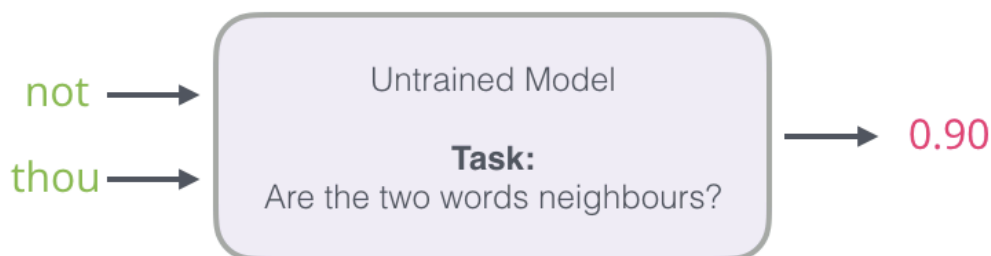
В этой статье сосредоточимся на первом шаге. Для увеличения производительности можно отойти от прогнозирования соседнего слова...

Change Task from



... и переключиться на модель, которая берёт входное и выходное слова и вычисляет вероятность их соседства (от 0 до 1).

To:



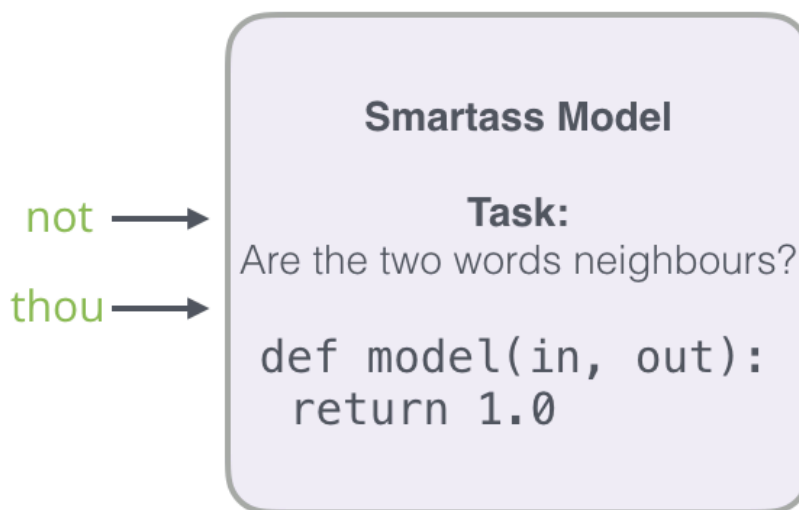
Такой простой переход заменяет нейронную сеть на модель логистической регрессии — таким образом, вычисления становятся намного проще и быстрее.

При этом требуется доработка структуры нашего набора данных: метка теперь является новым столбцом со значениями 0 или 1. В нашей таблице везде единицы, потому что мы добавляли туда соседей.

input word	target word
not	thou
not	shalt
not	make
not	a
make	shalt
make	not
make	a
make	machine

input word	output word	target
not	thou	1
not	shalt	1
not	make	1
not	a	1
make	shalt	1
make	not	1
make	a	1
make	machine	1

Такая модель вычисляется с невероятной скоростью: миллионы образцов за считанные минуты. Но нужно закрыть одну лазейку. Если все наши примеры положительные (цель: 1), то может образоваться хитрая модель, которая всегда возвращает 1, демонстрируя точность 100%, но она ничему не обучается и генерирует мусорные вложения.



Чтобы решить эту проблему, нужно ввести в набор данных *отрицательные образцы* — слова, которые точно не являются соседями. Для них модель обязана вернуть 0. Теперь модели придётся упорно работать, но вычисления по-прежнему идут на огромной скорости.

input word	output word	target
not	thou	1
not		0
not		0
not	shalt	1
not	make	1

➤ Negative examples

Для каждого образца в наборе данных добавляем отрицательные примеры с меткой 0

Но что ввести в качестве выходных слов? Выберем слова произвольно:

Pick randomly from vocabulary
(random sampling)

input word	output word	target
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	make	1

Word Count Probability

aardvark

aarhus

aaron

taco

thou

zyzzyva

Эта идея родилась под влиянием метода шумосопоставительного оценивания [pdf]. Мы сопоставляем фактический сигнал (положительные примеры соседних слов) с шумом (случайно выбранные слова, которые не являются соседями). Это обеспечивает отличный компромисс между производительностью и статистической эффективностью.

Skip-gram с отрицательной выборкой (SGNS)

Мы рассмотрели две центральные концепции word2vec: вместе они называются «skip-gram с отрицательной выборкой».

Skipgram

shalt	not	make	a	machine
input		output		
make		shalt		
make		not		
make		a		
make		machine		

Negative Sampling

input word	output word	target
make	shalt	1
make	aaron	0
make	taco	0

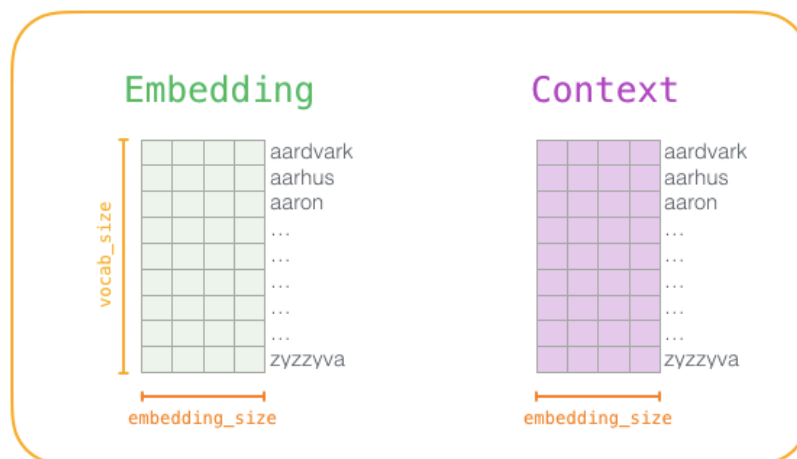
Обучение word2vec

«Машина не может предвидеть каждую проблему, важную для живого человека. Существует большая разница между дискретным пространством и непрерывным континуумом. Мы живём в одном пространстве, а машины существуют в другом». — *Бог-император Дюны*

Разобрав основные идеи skip-gram и отрицательной выборки, можем перейти к более пристальному рассмотрению процесса обучения word2vec.

Сначала предварительно обрабатываем текст, на котором обучаем модель. Определим размер словаря (будем называть его `vocab_size`), скажем, в 10 000 вложений и параметры слов в словаре.

В начале обучения создаём две матрицы: `Embedding` и `Context`. В этих матрицах хранятся вложения для каждого слова в нашем словаре (поэтому `vocab_size` является одним из их параметров). Второй параметр — размерность вложения (обычно `embedding_size` устанавливают на 300, но ранее мы рассматривали пример с 50 измерениями).

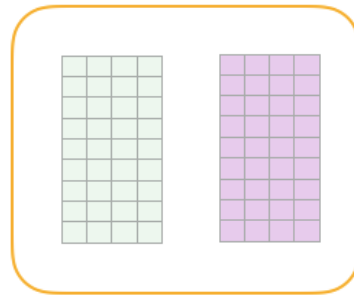


Сначала инициализируем эти матрицы случайными значениями. Затем начинаем процесс обучения. На каждом этапе берём один положительный пример и связанные с ним отрицательные. Вот наша первая группа:

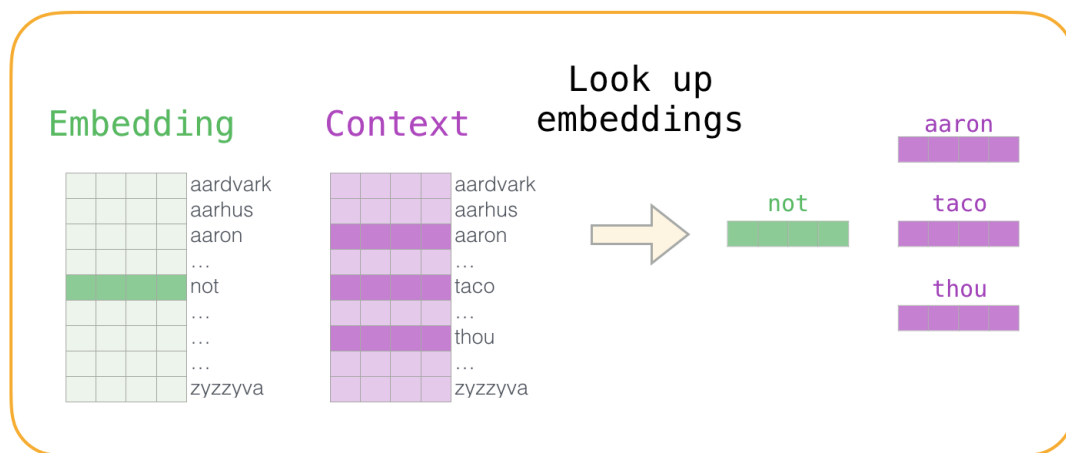
dataset

input word	output word	target
not	thou	1
not	aaron	0
not	taco	0
not	shalt	1
not	mango	0
not	finglonger	0
not	make	1
not	plumbus	0
...	...	...

model



Теперь у нас четыре слова: входное слово `not` и выходные/контекстные слова `thou` (фактический сосед), `aaron` и `taco` (отрицательные примеры). Начинаем поиск их вложений в матрицах `Embedding` (для входного слова) и `Context` (для контекстных слов), хотя в обеих матрицах есть вложения для всех слов из нашего словаря.



Затем вычисляем скалярное произведение входного вложения с каждым из контекстных вложений. В каждом случае получается число, которое указывает на сходство входных данных и контекстных вложений.

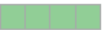
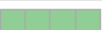
input word	output word	target	input • output
not	thou	1	0.2
not	aaron	0	-1.11
not	taco	0	0.74

Теперь нужен способ превратить эти оценки в некое подобие вероятностей: все они должны быть положительными числами между 0 и 1. Это отличная задача для логистических уравнений `sigmoid`.

input word	output word	target	input • output	sigmoid()
not	thou	1	0.2	0.55
not	aaron	0	-1.11	0.25
not	taco	0	0.74	0.68

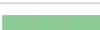
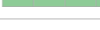
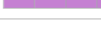
Результат вычисления sigmoid можно считать выдачей модели по этим образцам. Как видите, у taco самый высокий балл, а у aaron по-прежнему самая низкая оценка как до, так и после sigmoid.

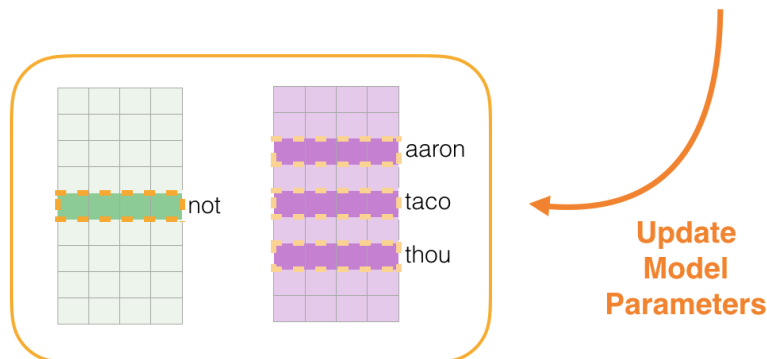
Когда необученная модель сделала прогноз и имея реальную целевую метку для сравнения, давайте посчитаем, сколько ошибок в прогнозе модели. Для этого просто вычитаем оценку sigmoid из целевых меток.

input word	output word	target	input • output	sigmoid()	Error
not 	thou 	1	0.2	0.55	0.45
not 	aaron 	0	-1.11	0.25	-0.25
not 	taco 	0	0.74	0.68	-0.68

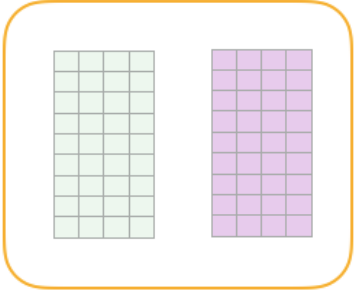
$error = target - sigmoid_scores$

Вот здесь наступает фаза «обучение» из термина «машинное обучение». Теперь мы можем использовать эту оценку ошибок для корректировки вложений not , thou , aaron и taco , чтобы при следующем расчёте результат был бы ближе к целевым оценкам.

input word	output word	target	input • output	sigmoid()	Error
not 	thou 	1	0.2	0.55	0.45
not 	aaron 	0	-1.11	0.25	-0.25
not 	taco 	0	0.74	0.68	-0.68



На этом завершается один этап обучения. Мы немного улучшили вложения нескольких слов (not , thou , aaron и taco). Теперь переходим к следующему этапу (следующий положительный образец и связанные с ним отрицательные) и повторяем процесс.

dataset			model	
input word	output word	target		
not	thou	1		
not	aaron	0		
not	taco	0		
not	shalt	1		
not	mango	0		
not	finglonger	0		
not	make	1		
not	plumbus	0		
...	...	...		

Вложения продолжают улучшаться, пока мы несколько раз циклически прогоняем весь набор данных. Затем можно остановить процесс, отложить матрицу Context и использовать обученную матрицу Embeddings для следующей задачи.

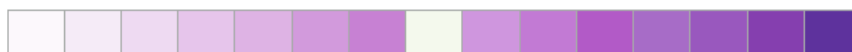
Размер окна и количество отрицательных образцов

В процессе обучения word2vec два ключевых гиперпараметра — это размер окна и количество отрицательных образцов.

Window size: 5



Window size: 15



Различные размеры окна подходят для разных задач. Замечено, что меньшие размеры окон (2–15) порождают *взаимозаменяемые* вложения с похожими индексами (обратите внимание, что антонимы часто взаимозаменяемы, если смотреть на окружающие слова: например, слова «хорошо» и «плохо» часто упоминаются в схожих контекстах). Большие размеры окон (15–50 или даже больше) порождают *родственные* вложения со схожими индексами. На практике вам часто придётся предоставлять аннотации ради полезного смыслового сходства в вашей задаче. В Gensim размер окна по умолчанию равен 5 (по два слова слева и справа, в дополнение к самому входному слову).

Negative samples: 2

input word	output word	target
make	shalt	1
make	aaron	0
make	taco	0

Negative samples: 5

input word	output word	target
make	shalt	1
make	aaron	0
make	taco	0
make	finglonger	0
make	plumbus	0
make	mango	0

Количество отрицательных образцов — ещё один фактор процесса обучения. Оригинальный документ рекомендует 5–20. В нём также говорится, что 2–5 образцов кажется достаточным, когда

у вас достаточно большой набор данных. В Gensim значение по умолчанию — 5 отрицательных образцов.

Вывод

«Если ваше поведение выпадает за ваши мерки, то вы живой человек, а не автомат» — *Бог-император Дюны*

Надеюсь, теперь вы поняли вложения слов и суть алгоритма word2vec. Также надеюсь, что теперь вам станут понятнее статьи, в которых упоминается концепция «skip-gram с отрицательной выборкой» (SGNS), как в вышеупомянутых рекомендательных системах.

Ссылки и дополнительная литература

- «Распределённые представления слов и фраз и их композиция» [pdf]
- «Эффективная оценка представлений слов в векторном пространстве» [pdf]
- «Нейронная вероятностная модель языка» [pdf]
- «Обработка речи и языка» Дэна Журафски и Джеймса Мартина — основной ресурс по NLP. Word2vec рассматривается в шестой главе.
- «Методы нейронных сетей в обработке естественного языка» by Йоава Голдберга — отличная книга по обработке естественного языка.
- Крис Маккормик написал несколько отличных постов в блоге о Word2vec. Он также только что выпустил электронную книгу «Внутренности word2vec»
- Хотите посмотреть код? Вот два варианта:
 - Реализация word2vec на Python в Gensim
 - Оригинальная реализация Миколова на C, а ещё лучше эта версия с подробными комментариями от Криса Маккормика
- Оценка распределительных моделей композиционной семантики
- «О вложениях слов», часть 2
- «Дюна»

Теги: NLP, Word2vec, embeddings, вложения, векторное представление слов, Gensim

Хабы: Машинное обучение

♦ +40

📖 372



💬 16

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Оставляя свою почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



1323

0

Карма Общий рейтинг

Анатолий Ализар @m1rko

автор, переводчик, редактор

Подписаться



Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



@gliderman

20 часов назад

Прокси для ленивых: поднимаем SOCKS5 поверх SSH, пока чайник закипает Простой 4 мин 15K

Тutorial

 +31 173 56

@atomnijpchelev

17 часов назад

Связь без мобильной сети: как я развернул свою VoIP-телефонию на участке Простой 4 мин 11K

Кейс

 +30 45 23

@TilekSamiev

22 часа назад

«Потерянные эпизоды» любимых сериалов в виде компьютерных игр Простой 12 мин 7.9K

Ретроспектива

 +25 19 1

@novoreg

1 час назад

Я задолбался читать про AI 4 мин 1.7K

Из песочницы

 +20 5 6

@TrexSelectel

5 часов назад

Как уход Crucial и ИИ-тренды влияют на цены DDR5 и SSD 5 мин 3.7K +16 1 6

@SlimRG

16 часов назад

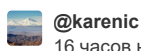
Чего нам стоит перевод фильма AI построить Средний 41 мин 7.3K

Тutorial

+16

37

6



@karenic

16 часов назад

Подключение SD карты по SPI (Капсула памяти)

Простой 11 мин 7.3K

Tutorial

+13

30

5



@Bright_Translate

4 часа назад

Несколько анекдотичных эпизодов из моей юности и ранней карьеры

Простой 7 мин 3.3K

Ретроспектива

Перевод

+11

8

2



@black_warlock_iv

16 часов назад

Убийца многомировой интерпретации квантовой механики

Сложный 7 мин 6.2K

Репортаж

+11

14

24



@foodofmen

4 часа назад

Эйджизм в IT: бороться нельзя скрывать

Простой 5 мин 4.3K

Из песочницы

+10

6

10

ИИ-ассистент в статьях на Хабре объяснит примеры с кодом

Турбо

Показать еще

МИНУТОЧКУ ВНИМАНИЯ



Кинозал и вид на залив: когда работа в офисе похожа на отпуск



Рейтинг IT-работодателей 2025: как и почему рынок штормит



Терраса + вид на залив = Ctrl+Alt+Вдох

ВАКАНСИИ

Embedded разработчик

от 127 500 Р · Алабуга · Москва

Программист C/C++ для Embedded-систем (Middle)

до 370 000 Р · Алабуга · Екатеринбург

Embedded Specialist (SDR / DSP)

от 127 500 Р · Алабуга · Казань

Инженер-разработчик встраиваемых систем (Embedded)

от 127 500 Р · Алабуга · Екатеринбург

Инженер-разработчик встраиваемых систем (Embedded)

от 127 500 Р · Алабуга · Томск

Больше вакансий на Хабр Карьере

ЧИТАЮТ СЕЙЧАС

Продам всё, что на фото. Недорого

👁 222K 💬 100

Конец эпохи паяльника или как гаражные энтузиасты случайно создали многомиллиардный бизнес

👁 31K 💬 40

Таймлайн: какие популярные иностранные сервисы заблокировал Роскомнадзор с 2016 года для пользователей в РФ

👁 29K 💬 57

Пользователи мобильного приложения «Госуслуги» на Android не могут зайти в госсервис без кода из Max

👁 141K 💬 463

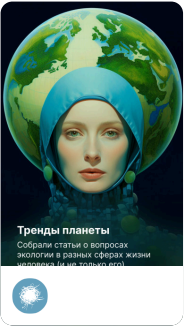
Математика боя: ученый из МФТИ построил модель современных военных действий

👁 7.9K 💬 20

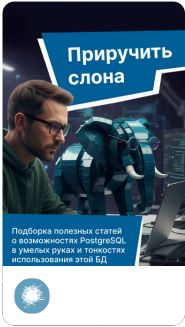
ИИ-ассистент в статьях на Хабре объяснит примеры с кодом

Турбо

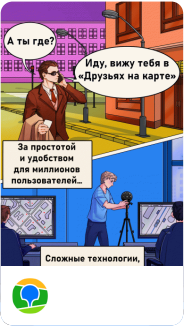
ИСТОРИИ



«Зелёная» подборка



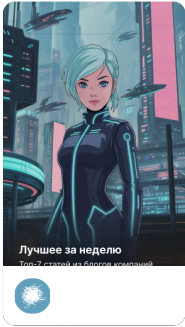
PostgreSQL каким вы его не знали



GPS врёт, а AR выпучает

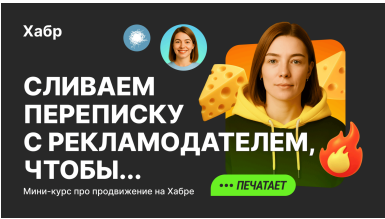


Настоящий Heavy Digital



Годные статьи от компаний

БЛИЖАЙШИЕ СОБЫТИЯ



23 сентября – 31 декабря

Бесплатный мини-курс: как продвигаться на Хабре, если вы — не IT-бренд

Онлайн

Маркетинг

Больше событий в календаре



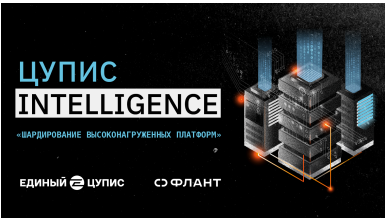
25 ноября 2025 – 16 января 2026

Сезон «ИИ в разработке»

Онлайн

Разработка Другое

Больше событий в календаре



9 декабря

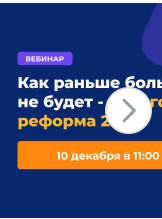
Митап ЕДИНОГО ЦУПИС и «Фланта»

«Шардирование высоконагруженных платформ»

Санкт-Петербург

Разработка Другое

Больше событий в календаре



10 декабря

Как раньше было — на реформа 2025

Онлайн

Другое

Больше событий

Ваш аккаунт	Разделы	Информация	Услуги
Войти	Статьи	Устройство сайта	Корпоративный блог
Регистрация	Новости	Для авторов	Медийная реклама
	Хабы	Для компаний	Нативные проекты
	Компании	Документы	Образовательные программы
	Авторы	Соглашение	Стартапам
	Песочница	Конфиденциальность	

Настройка языка

Техническая поддержка

